

EXPERIMENT NO. 3

Name of Student	<u>Manorath Ital</u>
Class Roll No	<u>19</u>
D.O.P.	<u>11-02-25</u>
D.O.S.	<u>18-02-25</u>
Sign and Grade	

AIM: To develop a basic Flask application with multiple routes and demonstrate the handling of GET and POST requests.

PROBLEM STATEMENT:

Design a Flask web application with the following features:

1. A homepage (/) that provides a welcome message and a link to a contact form.
 - a. Create routes for the homepage (/), contact form (/contact), and thank-you page (/thank_you).
2. A contact page (/contact) where users can fill out a form with their name and email.
3. Handle the form submission using the POST method and display the submitted data on a thank-you page (/thank_you).
 - a. On the contact page, create a form to accept user details (name and email).
 - b. Use the POST method to handle form submission and pass data to the thank-you page
4. Demonstrate the use of GET requests by showing a dynamic welcome message on the homepage when the user accesses it with a query parameter, e.g., /welcome?name=<user_name>.
 - a. On the homepage (/), use a query parameter (name) to display a personalized welcome message.

Theory:

1. Core Features of Flask:

Flask is a micro web framework for Python, popular for building lightweight and scalable web applications. It's known for its simplicity, flexibility, and ease of use.

Key Features:

- **Lightweight and Fast:** Flask doesn't include unnecessary modules by default, making it minimalistic and efficient.
- **Built-in Development Server:** Comes with a built-in server for local development and testing.
- **Modular & Extensible:** Easily integrates with external libraries such as Flask-SQLAlchemy and Flask-Login.
- **Templating with Jinja2:** Uses Jinja2 for creating dynamic HTML pages.
- **URL Routing:** Associates URLs with view functions using the `@app.route()` decorator.
- **RESTful Support:** Ideal for building RESTful APIs and services.
- **Session Handling:** Supports cookies and session management securely.
- **Debugging Tools:** Includes an interactive debugger and detailed error messages.

2. Why Use `Flask(__name__)`?

When creating a Flask app, the syntax `app = Flask(__name__)` is used. Here's why:

- `__name__` is a Python variable that tells Flask where to look for templates and static files.
- Helps determine whether the app is being run directly or imported elsewhere.
- Enables proper path resolution for static files, configurations, and logging.

3. What is a Template (Template Inheritance) in Flask?

In Flask, templates are HTML files rendered using **Jinja2**. Template inheritance allows developers to reuse common webpage elements like headers, footers, or navigation bars without repeating code.

- For example, instead of repeating `<html>`, `<head>`, or `<nav>` in every page, a base template is created and other pages inherit from it using `{% extends "base.html" %}` and `{% block content %}`.

4. HTTP Methods Supported by Flask:

Method	Description
GET	Retrieves data from the server.
POST	Sends new data to the server (e.g., form submission).
PUT	Updates existing resources.
DELETE	Removes a resource.
PATCH	Applies partial updates to data.

5. Difference Between Flask and Django:

Feature	Flask	Django
Framework Type	Micro-framework	Full-stack framework (batteries-included)
Flexibility	Highly customizable	Comes with many built-in features
Use Case	Best for small apps & APIs	Best suited for large, complex projects
Database Support	External ORMs like SQLAlchemy	Comes with built-in ORM

Admin Interface	No built-in admin panel	Built-in admin panel
Routing	Uses decorators like <code>@app.route()</code>	Uses <code>urls.py</code> file with path functions

6. Routing in Flask:

Routing links a URL path to a Python function.

python

Copy code

```
@app.route('/')
def home():
    return "Welcome to the Homepage!"
```

7. URL Building:

Flask's `url_for()` generates dynamic URLs, making code cleaner and more maintainable.

python

Copy code

```
@app.route('/profile/<username>')
def profile(username):
    return f"Hello, {username}!"

# Example usage
with app.test_request_context():
    print(url_for('profile', username='Alex')) # Output:
/profile/Alex
```

8. GET Request:

A **GET** request fetches data using URL parameters.

python

Copy code

```
@app.route('/welcome')
def welcome():
    name = request.args.get('name', 'Guest')
    return f"Welcome, {name}!"
```

9. POST Request:

A **POST** request is used to send form data to the server.

python

Copy code

```
@app.route('/submit', methods=['POST'])
def submit():
    name = request.form['name']
    return f"Thanks for submitting, {name}!"
```

Github Link:- <https://github.com/MANO-RATH/Webx3.git>

INPUT:-

Style.css

/* General Styling */

```
body {
    font-family: 'Poppins', 'Segoe UI', sans-serif;
    background: linear-gradient(135deg, #FF6B6B 0%, #4ECDC4 100%);
    margin: 0;
    padding: 0;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    flex-direction: column;
    color: #2d3436;
}
```

/* Container */

```
.container {
    background: rgba(255, 255, 255, 0.95);
    padding: 40px;
    border-radius: 20px;
    box-shadow: 0 15px 35px rgba(0, 0, 0, 0.2);
    width: 450px;
    text-align: center;
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.3);
    animation: fadeIn 0.5s ease-in-out;
```

```

}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(-20px); }
  to { opacity: 1; transform: translateY(0); }
}

h1 {
  color: #2d3436;
  margin-bottom: 25px;
  font-size: 2.4em;
  font-weight: 700;
  background: linear-gradient(45deg, #FF6B6B, #4ECDC4);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.1);
}

/* Buttons */
.btn {
  background: linear-gradient(45deg, #FF6B6B, #FF8E8E);
  color: white;
  border: none;
  padding: 14px 30px;
  border-radius: 12px;
  cursor: pointer;
  font-size: 16px;
  transition: all 0.3s ease;
  text-transform: uppercase;
  letter-spacing: 1.5px;
  font-weight: 600;
  box-shadow: 0 4px 15px rgba(255, 107, 107, 0.3);
}

.btn:hover {
  transform: translateY(-3px);
  box-shadow: 0 8px 25px rgba(255, 107, 107, 0.4);
  background: linear-gradient(45deg, #FF8E8E, #FF6B6B);
}

a {
  text-decoration: none;
  color: white;
}

a:hover {
  opacity: 0.9;
}

/* Form Styling */
form {

```

```

    text-align: left;
    margin-top: 20px;
}

label {
    display: block;
    margin-bottom: 8px;
    font-weight: 600;
    color: #2d3436;
    font-size: 1.1em;
}

input {
    width: 100%;
    padding: 14px;
    margin: 8px 0 20px;
    border: 2px solid #dfe6e9;
    border-radius: 12px;
    font-size: 16px;
    transition: all 0.3s ease;
    box-sizing: border-box;
    background: rgba(255, 255, 255, 0.9);
}

input:focus {
    border-color: #FF6B6B;
    outline: none;
    box-shadow: 0 0 0 3px rgba(255, 107, 107, 0.1);
}

input:hover {
    border-color: #b2bec3;
}

/* Footer */
.footer {
    margin-top: 30px;
    font-size: 14px;
    color: rgba(255, 255, 255, 0.9);
    text-shadow: 0 1px 2px rgba(0, 0, 0, 0.2);
    font-weight: 500;
}

/* Thank You Page Specific */
.thank-you-message {
    background: rgba(255, 255, 255, 0.8);
    padding: 25px;
    border-radius: 15px;
    margin: 25px 0;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
}

```

```
.thank-you-message p {  
  margin: 15px 0;  
  line-height: 1.8;  
  font-size: 1.1em;  
}
```

```
strong {  
  color: #FF6B6B;  
  font-weight: 700;  
}
```

Contact.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Join ConnectHub</title>  
  <link rel="stylesheet" href="/static/styles.css">  
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"  
rel="stylesheet">  
</head>  
<body>  
  <div class="container">  
    <h1>Join Our Community</h1>  
    <p class="form-intro">Fill out the form below to become part of our growing network of  
professionals and enthusiasts.</p>  
    <form action="/contact" method="post">  
      <label for="name">Your Full Name</label>  
      <input type="text" id="name" name="name" placeholder="Enter your name" required>  
  
      <label for="email">Professional Email</label>  
      <input type="email" id="email" name="email" placeholder="Enter your email" required>  
  
      <button type="submit" class="btn">Join Now</button>  
    </form>  
  </div>  
  <div class="footer">ConnectHub &copy; 2024 - Building Bridges, One Connection at a Time</div>  
</body>  
</html>
```

Index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Welcome to ConnectHub</title>  
  <link rel="stylesheet" href="/static/styles.css">
```



```

    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"
rel="stylesheet">
</head>
<body>
    <div class="container">
        <h1>Hello, {{ name }}! 🙌</h1>
        <p class="welcome-text">Welcome to ConnectHub - Your Gateway to Seamless
Communication</p>
        <p class="sub-text">Join our community and stay connected with the latest updates and
opportunities.</p>
        <a href="/contact" class="btn">Get Started</a>
    </div>
    <div class="footer">ConnectHub &copy; 2024 - Building Bridges, One Connection at a Time</div>
</body>
</html>

```

Thank_you.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Welcome to ConnectHub</title>
    <link rel="stylesheet" href="/static/styles.css">
    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"
rel="stylesheet">
</head>
<body>
    <div class="container">
        <h1>Welcome Aboard! 🎉</h1>
        <div class="thank-you-message">
            <p>Hey <strong>{{ name }}</strong>, we're thrilled to have you join our community!</p>
            <p>We've sent a welcome email to <strong>{{ email }}</strong> with all the details about your
new ConnectHub membership.</p>
            <p>Get ready to connect, collaborate, and grow with professionals from around the world!</p>
        </div>
        <a href="/" class="btn">Explore ConnectHub</a>
    </div>
    <div class="footer">ConnectHub &copy; 2024 - Building Bridges, One Connection at a Time</div>
</body>
</html>

```

app.py

```

from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def home():
    name = request.args.get('name', 'Guest')
    return render_template('index.html', name=name)

@app.route('/contact', methods=['GET', 'POST'])

```

```
def contact():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        return render_template('thank_you.html', name=name, email=email)
    return render_template('contact.html')

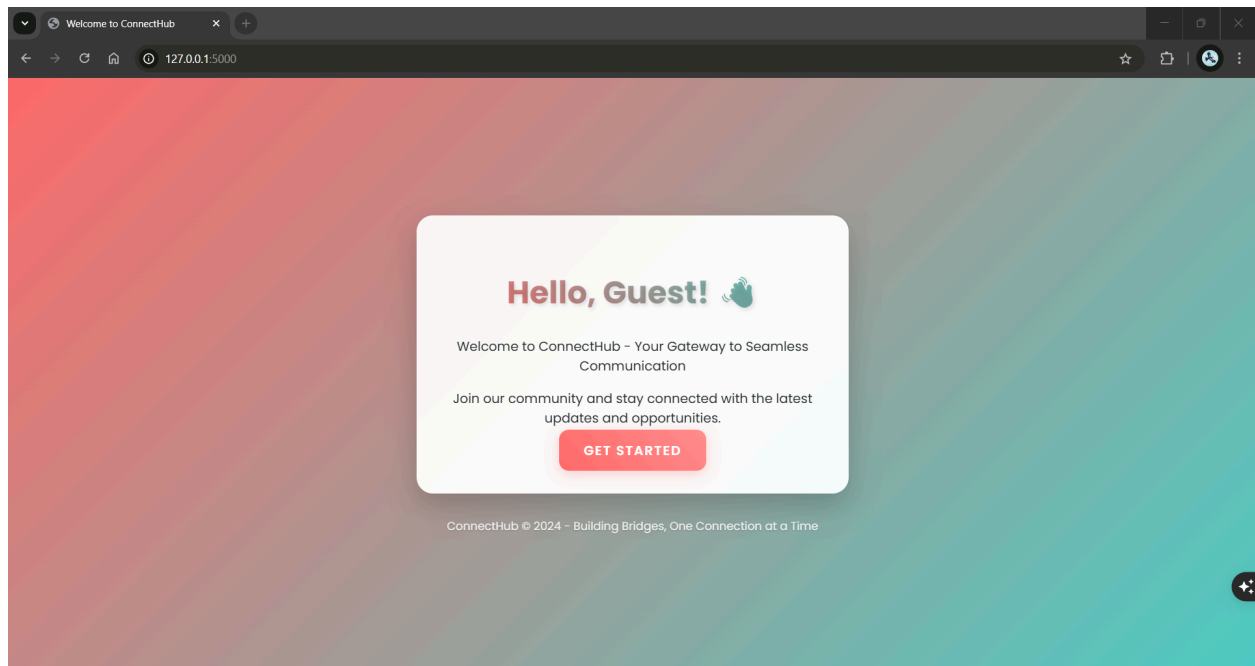
@app.route('/thank_you')
def thank_you():
    return "Thank you for your submission!"
```

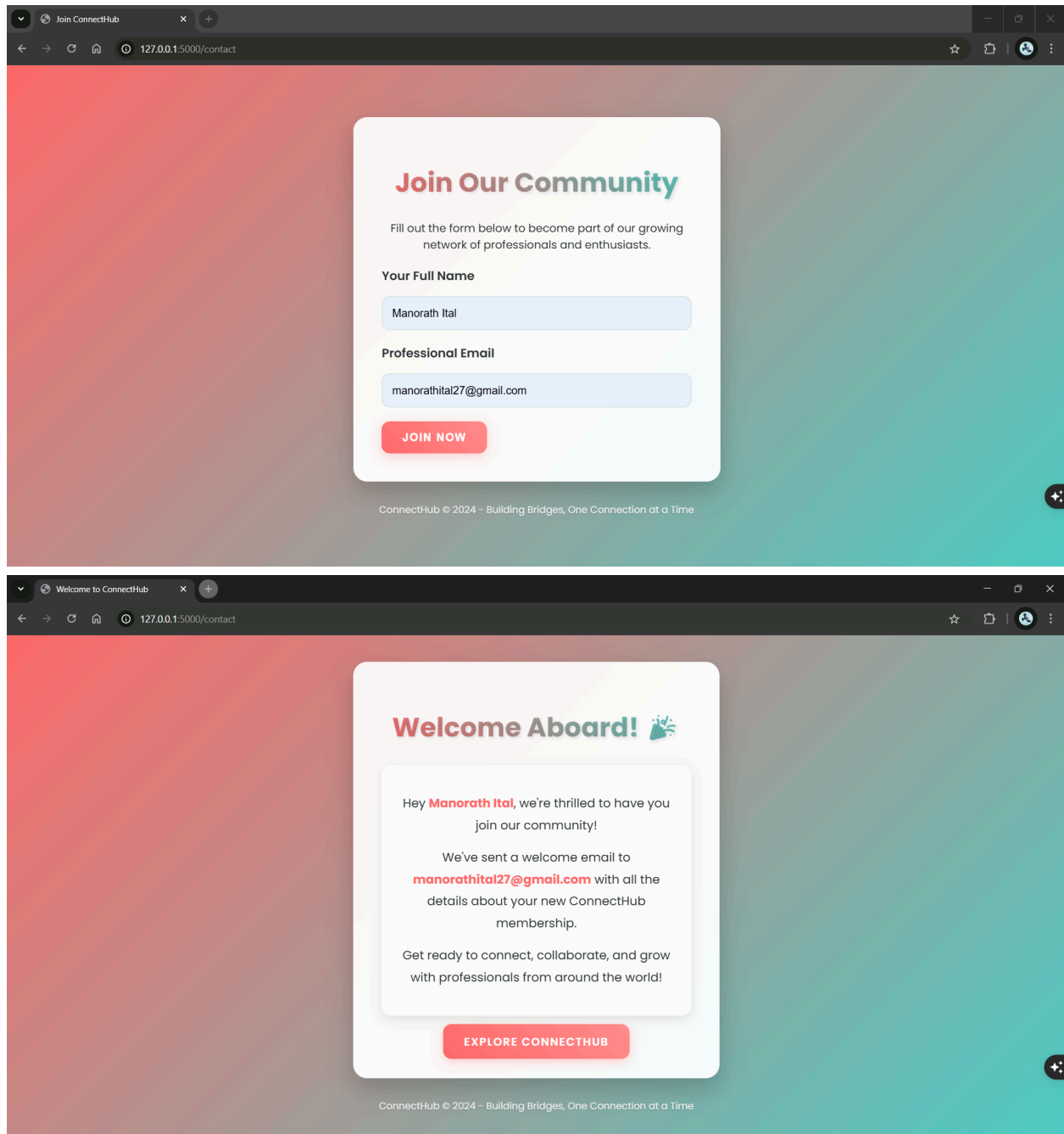
```
if __name__ == '__main__':
    app.run(debug=True)
```

Requirements.txt

Flask==3.0.0

OUTPUT : -





Conclusion: -

The Flask application successfully demonstrates handling multiple routes and processing both GET and POST requests. The homepage dynamically personalizes the welcome message using query parameters, while the contact form collects user details and submits them via POST. The submitted data is then displayed on a thank-you page, showcasing form handling and data transfer between routes. This implementation highlights Flask's ability to create interactive web applications with efficient request handling and user input management.